WEEK 02 — Registers, Register Transfer Language (RTL) & Micro-Operations

Complete Notes
English Only
Theory + Numericals

This week covers two major lectures:

1. Registers and Register Transfer Language (RTL)
2. Arithmetic, Logic and Shift Micro-Operations

These topics are important because they explain how data moves and gets processed inside the CPU.

PART 1: Registers and Their Purpose

1. What is a Register?

Definition:

A register is a small, high-speed storage location inside the CPU that temporarily stores data, instructions, memory addresses, and intermediate results during program execution.

Explanation

Registers are the fastest storage locations available to the processor. They hold information that the CPU needs immediately while executing instructions.

For example, consider this operation:

$$ A = 10 + 20 $$

The CPU may store:

- 10 in Register R1.
- 20 in Register R2.
- The addition result (30) in Register R3.

Registers help the CPU access data quickly without repeatedly accessing main memory.

Characteristics of Registers

1. Registers are located inside the CPU.
2. They have very high access speed.
3. They have limited storage capacity.
4. They temporarily hold data and instructions.
5. They are used during instruction execution.
6. Each register has a specific size, such as 8, 16, 32, or 64 bits.

2. Why Are Registers Important?

Registers are important because they:

1. Store operands required for calculations.
2. Hold intermediate processing results.
3. Store memory addresses.
4. Temporarily hold instructions.
5. Reduce the need for frequent memory access.
6. Improve CPU processing speed.

Example:

When the CPU executes:

\[R3 \leftarrow R1 + R2 \]

It takes the values from R1 and R2, adds them, and stores the result in R3.

3. Types of Registers

Registers are generally classified into two major categories:

[The Essential Guide to CPU Registers: Types, Functions, and Performance | The Silicon Journal
medium.com]

A. General-Purpose Registers (GPR)

Definition:

General-purpose registers are CPU registers used to store temporary data, operands, and intermediate results during program execution.

They can be used for different operations depending on the instruction.

Examples:

- AX

- BX
- CX
- DX

Functions:

Register	Common purpose in 8086
AX	Accumulator; arithmetic and I/O operations
BX	Base register; addressing
CX	Counter for loops and shifts
DX	Data register; multiplication, division and I/O

Example:

```
MOV AX, 10
MOV BX, 20
ADD AX, BX
```

Explanation:

1. `MOV AX, 10` stores 10 in AX.
2. `MOV BX, 20` stores 20 in BX.
3. `ADD AX, BX` adds the values.
4. AX becomes 30.

We will study these registers in more detail in Week 7.

B. Special-Purpose Registers

Definition:

Special-purpose registers are registers designed to perform specific control, addressing, instruction management, or data-handling functions inside the CPU.

Important examples:

Register	Function
PC (Program Counter)	Holds the address of the next instruction
IR (Instruction Register)	Holds the current instruction being executed
MAR (Memory Address Register)	Holds the memory address to be accessed
MDR (Memory Data Register)	Holds data being transferred to or from memory
SP (Stack Pointer)	Holds the address of the top of the stack

Important Conceptual Questions

Q1. Why are registers faster than RAM?

Registers are located inside the CPU and are designed for immediate access during instruction execution. RAM is a separate main memory that requires memory access operations.

Q2. What is the difference between general-purpose and special-purpose registers?

General-purpose registers store temporary data and operands for different operations, whereas special-purpose registers perform specific functions such as instruction sequencing, memory addressing, and stack management.

Q3. What is the function of the Program Counter?

The Program Counter holds the address of the next instruction that the CPU will fetch.

Q4. What is the difference between PC and IR?

- PC holds the address of the next instruction.
- IR holds the current instruction being executed.

PART 2: Register Transfer

4. What is Register Transfer?

Definition:

Register Transfer is the process of moving binary information from one register to another register within a computer system.

Explanation

A CPU contains multiple registers. During instruction execution, data frequently moves between these registers.

For example, if we want to copy the contents of register R1 into register R2, we perform a register transfer.

\[R2 \leftarrow R1 \]

This means the contents of R1 are copied into R2.

Important: The original contents of R1 remain unchanged.

Example

Suppose:

$$ R1 = 1010 $$
$$ R2 = 0000 $$

After performing:

$$ R2 \leftarrow R1 $$

The result becomes:

$$ R1 = 1010 $$
$$ R2 = 1010 $$

R1 remains unchanged, while R2 receives a copy of its contents.

5. What is Register Transfer Language (RTL)?

Very Important

Definition:

Register Transfer Language (RTL) is a symbolic notation used to describe the transfer of information between registers and the micro-operations performed on the data stored in registers.

Explanation

RTL is used to represent internal CPU operations in a simple mathematical form.

Instead of writing lengthy explanations, we use symbols to describe what happens.

For example:

$$ R2 \leftarrow R1 $$

This represents the transfer of data from R1 to R2.

Common RTL Symbols

Symbol	Meaning
←	Transfer or assignment
,	Separates operations
+	Addition
−	Subtraction

AND	Logical AND
OR	Logical OR
XOR	Exclusive OR
NOT	Complement
SHL	Shift left
SHR	Shift right
INC	Increment
DEC	Decrement

Important Concept

The arrow does not mean mathematical equality.

For example:

$$ R2 \leftarrow R1 $$

It means the contents of R1 are transferred into R2, not that R1 and R2 were already equal.

6. Basic Register Transfer Examples

Example 1: Transfer from R1 to R2

$$ R2 \leftarrow R1 $$

Meaning: Copy the contents of R1 into R2.

Example 2: Transfer from R3 to R4

$$ R4 \leftarrow R3 $$

Meaning: Copy the contents of R3 into R4.

Example 3: Addition

$$ R3 \leftarrow R1 + R2 $$

Meaning:

- Take contents of R1.
- Take contents of R2.
- Add both values.
- Store the result in R3.

Example 4: Subtraction

\[R3 \leftarrow R1 - R2 \]

Meaning: Subtract the contents of R2 from R1 and store the result in R3.

Example 5: Conditional Register Transfer

\[P: R2 \leftarrow R1 \]

Meaning: Transfer the contents of R1 into R2 only when control condition P is true.

Here, P represents a control condition.

Important: Conditional transfers are performed only when the specified condition is satisfied.

PART 3: Micro-Operations

7. What is a Micro-Operation?

Definition:

A micro-operation is an elementary operation performed on the data stored in registers during one or more clock cycles of CPU operation.

Explanation

A computer instruction may require several small internal operations.

These small operations are called micro-operations.

For example, consider:

\[R3 \leftarrow R1 + R2 \]

The CPU performs the following basic operations:

1. Read the contents of R1.
2. Read the contents of R2.
3. Perform addition using the ALU.
4. Store the result in R3.

These internal operations are described using RTL.

Characteristics of Micro-Operations

1. They operate on data stored in registers.
2. They are basic internal CPU operations.
3. They are controlled by control signals.
4. They are performed during instruction execution.
5. They can involve data transfer, arithmetic, logic, or shifting.

8. Types of Micro-Operations

There are four main types:

Micro operations | PPT
slideshare.net

1. Register Transfer Micro-Operations
2. Arithmetic Micro-Operations
3. Logic Micro-Operations
4. Shift Micro-Operations

Let's study each one.

PART 4: Arithmetic Micro-Operations

9. What are Arithmetic Micro-Operations?

Definition:

Arithmetic micro-operations are operations performed on numerical data stored in registers using arithmetic operations such as addition, subtraction, increment, and decrement.

They are generally performed by the ALU.

Types of Arithmetic Micro-Operations

A. Addition

Addition combines two binary numbers and stores the result in a destination register.

RTL:

\[R3 \leftarrow R1 + R2 \]

Example:

Given:

$$ R1 = 0101 $$
$$ R2 = 0011 $$

Perform:

$$ R3 \leftarrow R1 + R2 $$

Solution:

```
   0101
 + 0011
 -------
   1000
```

Answer: 1000 (Binary) = 8 (Decimal)

B. Subtraction

Subtraction calculates the difference between two register values.

RTL:

$$ R3 \leftarrow R1 - R2 $$

Example:

Given:

$$ R1=1000 $$
$$ R2=0011 $$

Solution:

```
   1000
 - 0011
 -------
   0101
```

Answer: 0101 = 5

C. Increment

Definition:

Increment is an arithmetic micro-operation that increases the value of a register by one.

RTL:

$$ R1 \leftarrow R1+1 $$

Example:

Given:

$$ R1=0101 $$

After increment:

$$ 0101+0001=0110 $$

Answer = 0110 (6 in decimal).

D. Decrement

Definition:

Decrement is an arithmetic micro-operation that decreases the value of a register by one.

RTL:

$$ R1 \leftarrow R1-1 $$

Example:

Given:

$$ R1=0101 $$

After decrement:

$$ 0101-0001=0100 $$

Answer = 0100 (4 in decimal).

E. Negation

Definition:

Negation changes the sign of a number by taking its two's complement representation.

For an n-bit binary number:

1. Invert all bits.
2. Add 1.

Example:

Find the two's complement of 00000101.

Step 1: Invert bits.

```
00000101
11111010
```

Step 2: Add 1.

```
11111010
+       1
---------
11111011
```

Answer:

\[-5 = 11111011 \]

(For 8-bit two's complement representation.)

Summary of Arithmetic Micro-Operations

Operation	RTL
Addition	\(R3 \leftarrow R1+R2\)
Subtraction	\(R3 \leftarrow R1-R2\)
Increment	\(R1 \leftarrow R1+1\)
Decrement	\(R1 \leftarrow R1-1\)
Negation	\(R1 \leftarrow -R1\)

PART 5: Logic Micro-Operations

10. What are Logic Micro-Operations?

Definition:

Logic micro-operations are operations performed on the individual bits of data stored in registers using logical operations such as AND, OR, XOR, and NOT.

Explanation

Logic micro-operations work directly on binary bits (0 and 1).

Unlike arithmetic operations, they do not perform ordinary mathematical calculations.

They manipulate individual bits.

Types of Logic Micro-Operations

A. AND Operation

AND produces 1 only when both corresponding bits are 1.

RTL:

\[R3 \leftarrow R1 \ AND\ R2 \]

Example:

```
R1 = 1010
R2 = 1100
------------
     1000
```

Explanation:

R1	R2	Result
1	1	1
0	1	0
1	0	0
0	0	0

Answer = 1000.

B. OR Operation

OR produces 1 when at least one corresponding bit is 1.

RTL:

\[R3 \leftarrow R1\ OR\ R2 \]

Example:

```
R1 = 1010
R2 = 1100
------------
     1110
```

C. XOR Operation

XOR produces 1 when the corresponding bits are different.

RTL:

\[R3 \leftarrow R1\ XOR\ R2 \]

Example:

```
R1 = 1010
R2 = 1100
------------
     0110
```

D. NOT Operation

NOT reverses every bit.

RTL:

\[R2 \leftarrow NOT\ R1 \]

Example:

```
R1 = 1010
------------
R2 = 0101
```

Important Truth Table

Memorize this table for the exam.

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

NOT Truth Table:

A	NOT A
0	1
1	0

Applications of Logic Micro-Operations

1. AND is used for masking selected bits.
2. OR is used for setting selected bits.
3. XOR is used for toggling bits.
4. NOT is used for complementing bits.

Conceptual Questions

Q1. Which operation produces 1 only when both bits are 1?

AND.

Q2. Which operation produces 1 when inputs are different?

XOR.

Q3. Which operation reverses all bits?

NOT.

Q4. What is the difference between arithmetic and logic micro-operations?

Arithmetic micro-operations perform numerical calculations such as addition and subtraction, whereas logic micro-operations manipulate individual binary bits using AND, OR, XOR, and NOT.

PART 6: Shift Micro-Operations

11. What are Shift Micro-Operations?

Definition:

Shift micro-operations move the bits of a register to the left or right by one or more positions.

Explanation

In shift operations, binary bits change their positions inside a register.

There are three major types:

1. Logical Shift
2. Circular (Rotate) Shift
3. Arithmetic Shift

Solved Consider the design of a 4-bit combinational circuit | Chegg.com
chegg.com

12. Logical Shift

Definition:

Logical shift moves all bits in a register to the left or right, inserting zero into the empty position.

A. Logical Shift Left (SHL)

In Logical Shift Left:

- Every bit moves one position to the left.
- The leftmost bit is discarded.
- A zero enters the rightmost position.

Example:

Given:

```
R1 = 00110101
```

Perform one-bit logical shift left:

```
Before: 00110101
After:  01101010
```

RTL:

\[R1 \leftarrow SHL(R1) \]

Important: For unsigned binary numbers, shifting left by one position is equivalent to multiplying by 2, provided no significant bit is lost.

Example:

\[00000101=5 \]

After shifting left:

\[00001010=10 \]

B. Logical Shift Right (SHR)

In Logical Shift Right:

- Every bit moves one position to the right.
- The rightmost bit is discarded.
- A zero enters the leftmost position.

Example:

```
Before: 00110101
After:  00011010
```

RTL:

\[R1 \leftarrow SHR(R1) \]

For unsigned numbers, shifting right by one position is equivalent to integer division by 2.

Example:

\[00001010=10 \]

After shifting right:

\[00000101=5 \]

13. Circular Shift (Rotate)

Definition:

Circular shift is a micro-operation in which the bit shifted out from one end of the register is inserted back into the other end.

Unlike logical shift, no bit is permanently lost.

A. Circular Shift Left (ROL)

The leftmost bit moves to the rightmost position.

Example:

```
Before: 10110001
After:  01100011
```

Explanation:

- The leftmost bit (1) is removed from the left.
- Remaining bits shift left.
- The removed bit is inserted at the right.

B. Circular Shift Right (ROR)

The rightmost bit moves to the leftmost position.

Example:

```
Before: 10110001
After:  11011000
```

Explanation:

- The rightmost bit (1) is removed.
- Remaining bits shift right.
- The removed bit is inserted at the left.

14. Arithmetic Shift

Definition:

Arithmetic shift is a shift operation used with signed binary numbers while preserving the sign of the number during right shifts.

It is mainly used for signed integer operations.

A. Arithmetic Shift Right (ASR)

In Arithmetic Shift Right:

- All bits move one position right.
- The sign bit (MSB) is copied into the new leftmost position.
- The rightmost bit is discarded.

Example:

```
Before: 11101000
After:  11110100
```

The leftmost bit is 1, so 1 is copied into the new leftmost position.

Why?

Because in two's complement representation, the most significant bit represents the sign:

- 0 = Positive
- 1 = Negative

B. Arithmetic Shift Left (ASL)

In Arithmetic Shift Left:

- All bits move one position left.
- The rightmost position becomes 0.
- The leftmost bit is discarded.

Example:

```
Before: 00101000
After:  01010000
```

Important: Arithmetic shift left can cause signed overflow. The sign may change if the result exceeds the representable range.

Comparison of Shift Operations

Operation	What happens to shifted-out bit?	New bit
Logical Left	Discarded	0 at right
Logical Right	Discarded	0 at left
Circular Left	Reinserted at right	Old MSB
Circular Right	Reinserted at left	Old LSB
Arithmetic Right	Discarded	Old sign bit
Arithmetic Left	Discarded	0 at right

Very Important Conceptual Questions

Q1. What is the difference between logical and arithmetic right shift?

Logical right shift inserts zero at the leftmost position, whereas arithmetic right shift copies the original sign bit into the leftmost position to preserve the sign of a signed number.

Q2. What is the difference between logical and circular shift?

Logical shift discards the shifted-out bit and inserts zero, whereas circular shift reinserts the shifted-out bit at the opposite end.

Q3. Why is arithmetic shift used?

It is used for signed binary numbers, particularly for operations equivalent to multiplication or division by powers of two, subject to overflow and rounding considerations.

Q4. Which shift operation preserves the sign bit?

Arithmetic shift right.

PART 7: Important Solved Numerical Questions

Numerical 1: Register Addition

Given:

$$ R1=1010 $$
$$ R2=0101 $$

Find:

$$ R3\leftarrow R1+R2 $$

Solution:

```
   1010
 + 0101
 -------
   1111
```

Answer:

$$ R3=1111 $$

Numerical 2: Register Subtraction

Given:

$$ R1=1100 $$
$$ R2=0100 $$

Find:

$$ R3\leftarrow R1-R2 $$

Solution:

```
   1100
 - 0100
 -------
   1000
```

Answer:

\[R3=1000 \]

Numerical 3: Increment

Given:

\[R1=0111 \]

Perform:

\[R1\leftarrow R1+1 \]

Solution:

```
   0111
 + 0001
 -------
   1000
```

Answer = 1000.

Numerical 4: AND, OR and XOR

Given:

\[R1=1100 \]
\[R2=1010 \]

Find all three operations.

AND:

```
  1100
  1010
  ----
  1000
```

OR:

```
  1100
  1010
  ----
  1110
```

XOR:

```
  1100
  1010
  ----
  0110
```

Numerical 5: NOT Operation

Given:

\[R1=10110100 \]

Find the complement.

Solution:

```
Original: 10110100
NOT:      01001011
```

Answer:

\[R2=01001011 \]

Numerical 6: Logical Shift Left

Given:

\[R1=10110010 \]

Perform one-bit SHL.

Solution:

```
Before: 10110010
After:  01100100
```

Answer:

\[01100100 \]

Numerical 7: Logical Shift Right

Given:

\[R1=10110010 \]

Perform one-bit SHR.

Solution:

```
Before: 10110010
After:  01011001
```

Answer:

\[01011001 \]

Numerical 8: Circular Shift Left

Given:

\[R1=10110010 \]

Perform one-bit circular left shift.

Solution:

```
Before: 10110010
After:  01100101
```

Answer:

\[01100101 \]

Numerical 9: Circular Shift Right

Given:

\[R1=10110010 \]

Perform one-bit circular right shift.

Solution:

```
Before: 10110010
After:  01011001
```

Answer:

\[01011001 \]

Notice that this example has a rightmost bit of 0, so circular right shift and logical right shift produce the same result.

Numerical 10: Arithmetic Shift Right

Given:

$$ R1=11101000 $$

Perform one-bit arithmetic right shift.

Solution:

```
Before: 11101000
After:  11110100
```

Answer:

$$ 11110100 $$

The sign bit (1) is copied into the leftmost position.

PART 8: Important Differences for Exam

1. Register vs Memory

Register	Memory
Located inside CPU.	Main memory is separate from CPU.
Very high speed.	Generally slower.
Small capacity.	Larger capacity.
Holds immediate data.	Holds programs and active data.

2. Arithmetic vs Logic Micro-Operations

Arithmetic	Logic
Performs mathematical operations.	Performs bitwise operations.
Addition and subtraction.	AND, OR, XOR, NOT.
Works on numerical values.	Works on individual bits.
Example: R3 ← R1 + R2	Example: R3 ← R1 AND R2

3. Logical vs Circular vs Arithmetic Shift

Logical Shift	Circular Shift	Arithmetic Shift
Inserts zero.	Reinserts shifted-out bit.	Preserves sign during right shift.

Used for bit manipulation.	Used for bit rotation.	Used for signed numbers.
Shifted-out bit is discarded.	Shifted-out bit is reused.	Rightmost bit is discarded in right shift.

4. Register Transfer vs Micro-Operation

Register Transfer	Micro-Operation
Moves data between registers.	Performs basic operations on register data.
Example: R2 ← R1	Example: R3 ← R1 + R2
Focuses on data movement.	Includes transfer, arithmetic, logic and shifting.

PART 9: Important Expected Exam Questions

Short Questions

1. Define a register.
2. Why are registers faster than RAM?
3. Define General-Purpose Registers.
4. Define Special-Purpose Registers.
5. What is Register Transfer?
6. What is Register Transfer Language?
7. What does the RTL arrow represent?
8. Define micro-operation.
9. List the four types of micro-operations.
10. Define arithmetic micro-operations.
11. Define logic micro-operations.
12. What is increment and decrement?
13. Define logical shift.
14. What is circular shift?
15. What is arithmetic shift?
16. What is the difference between PC and IR?
17. What is the function of MAR and MDR?

Long Questions

1. Explain registers and their types with examples.
2. Explain Register Transfer Language (RTL) with examples.
3. Define micro-operations and explain their four types.
4. Explain arithmetic micro-operations with examples.
5. Explain logic micro-operations with truth tables.
6. Explain shift micro-operations and their types with diagrams.
7. Differentiate between logical, circular and arithmetic shifts.

PART 10: Practice Test — Week 2

Try these questions without looking at the notes.

Week 2 MCQ Test

0/8 attempted

Q1. Which register holds the address of the next instruction?

IR
PC
MAR
MDR

Q2. Which RTL represents copying R1 into R2?

R1 ← R2
R2 ← R1
R1 + R2
R2 ← R1 + 1

Q3. Which micro-operation increases a register value by one?

Decrement
Negation
Increment
Complement

Q4. What is the result of 1010 AND 1100?

1110
0110
1000
1010

Q5. What is the result of 1010 XOR 1100?

1110
0110
1000
0100

Q6. In logical right shift, which bit enters the MSB position?

1
Old LSB
0
Old MSB

Q7. Which shift operation reinserts the shifted-out bit at the opposite end?

Logical shift
Circular shift
Arithmetic shift
None

Q8. Which register holds the current instruction?

PC
SP
IR
MAR
Submit Answers

WEEK 2 — Quick Revision Sheet

Important RTL expressions

Operation	RTL
Transfer	\(R2 \leftarrow R1\)
Addition	\(R3 \leftarrow R1+R2\)
Subtraction	\(R3 \leftarrow R1-R2\)
Increment	\(R1 \leftarrow R1+1\)
Decrement	\(R1 \leftarrow R1-1\)
AND	\(R3 \leftarrow R1 \ AND\ R2\)
OR	\(R3 \leftarrow R1 \ OR\ R2\)
XOR	\(R3 \leftarrow R1 \ XOR\ R2\)
NOT	\(R2 \leftarrow NOT\ R1\)
Shift Left	\(R1 \leftarrow SHL(R1)\)
Shift Right	\(R1 \leftarrow SHR(R1)\)

My exam preparation advice: Give special attention to RTL expressions, truth tables, binary addition/subtraction, AND/OR/XOR/NOT operations, and all three types of shift operations. These are especially suitable for numerical and practical questions.

You should be able to solve a micro-operation question even when the examiner changes the binary numbers or register names.